

Next.js y el App Router

Arquitectura Full-Stack, Renderizado Híbrido y
React Server Components

Computación en Internet 3 — Universidad Icesi

01. ¿Por qué Next.js?

De una biblioteca de interfaz (React) a un framework full-stack de producción con renderizado híbrido y optimización de recursos.

React vs Next.js: Librería vs Framework

Arquitectura Base: React gestiona el árbol de componentes en el cliente; Next.js proporciona el entorno de ejecución full-stack con enrutador, empaquetador Turbopack y optimizaciones de fábrica.

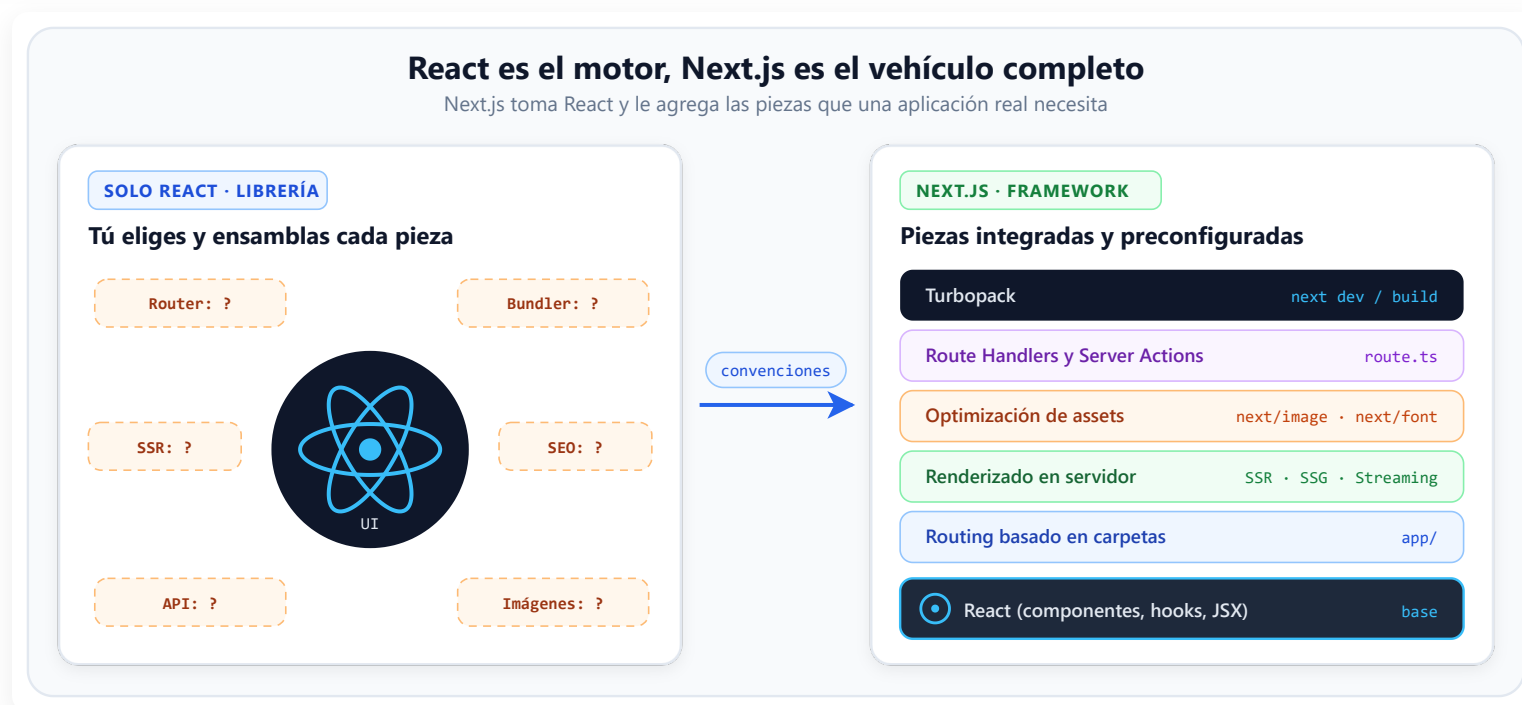
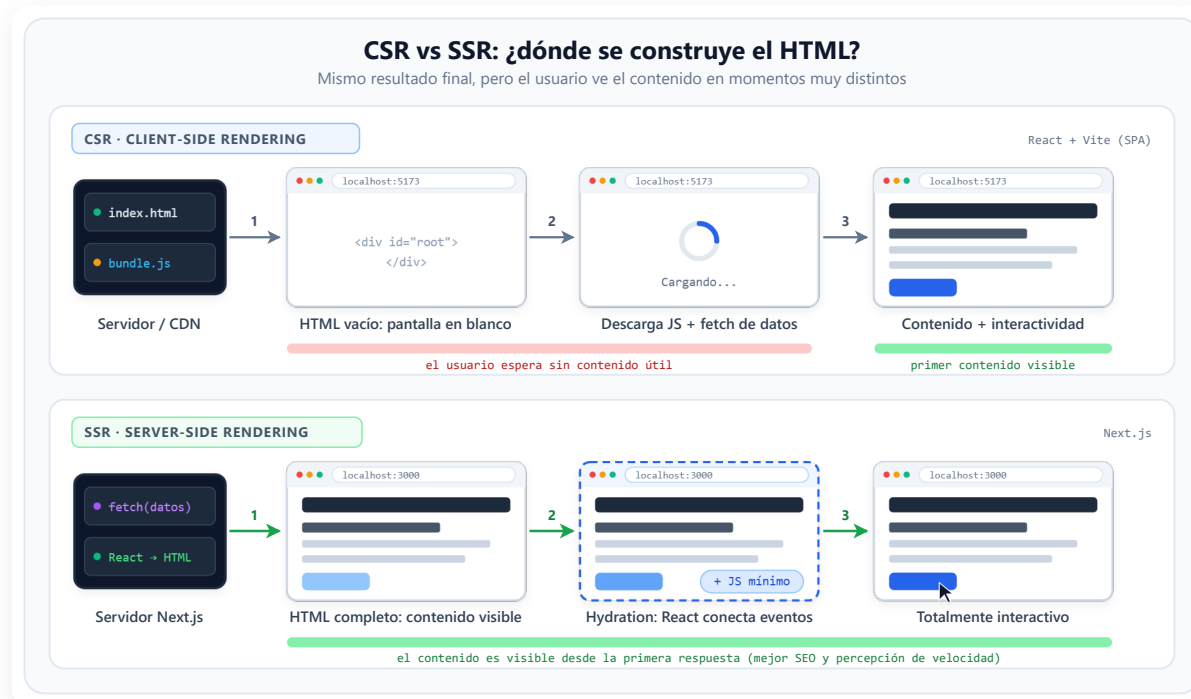


Tabla Comparativa: React vs Next.js

Característica	React (Librería UI)	Next.js (Framework Full-Stack)
Naturaleza	Biblioteca de vista (V en MVC) enfocada exclusivamente en componentes.	Framework integral: cliente, servidor Node.js y compilador integrados.
Enrutamiento	Requiere instalar librerías externas de terceros (ej. <code>react-router-dom</code>).	Enrutador nativo basado en carpetas (App Router en <code>app/</code>).
Estrategia de Render	Client-Side Rendering (CSR) por defecto; emite un HTML vacío <code><div id="root"></code> .	Renderizado híbrido: Server Components (SSR), SSG e interactividad cliente.
Obtención de Datos	En cliente vía <code>useEffect</code> + <code>fetch</code> (provoca waterfalls de red).	<code>async/await</code> directo en servidor con deduplicación y caché nativa.
Optimización de Assets	Configuración manual del empaquetador (Vite/Webpack) y compresión manual.	Automática con <code>next/image</code> (WebP/AVIF), <code>next/font</code> y scripts diferidos.
SEO e Indexabilidad	Complejo; los rastreadores web deben esperar la ejecución de JavaScript.	SEO garantizado: HTML semántico pre-renderizado y Metadata API nativa.

Estrategias de Renderizado: CSR vs SSR

Flujo de Entrega: En CSR el navegador recibe un HTML en blanco y espera la descarga de JS para pintar; en SSR el servidor pre-renderiza y entrega HTML visible desde el primer paquete de red.



¿Cuándo Usar SSR vs CSR? Guía de Decisión

SERVER-SIDE RENDERING (SSR / SERVER)

-  **SEO Crítico:** E-commerce, artículos, blogs y páginas públicas indexables.
-  **Móviles y Redes Lentas:** FCP inmediato; el usuario ve contenido sin esperar JS.
-  **Datos Dinámicos:** Páginas con información que cambia en cada petición.
-  **Seguridad Backend:** Acceso a bases de datos y API keys privadas sin exponerlas.

CLIENT-SIDE RENDERING (CSR / 'USE CLIENT')

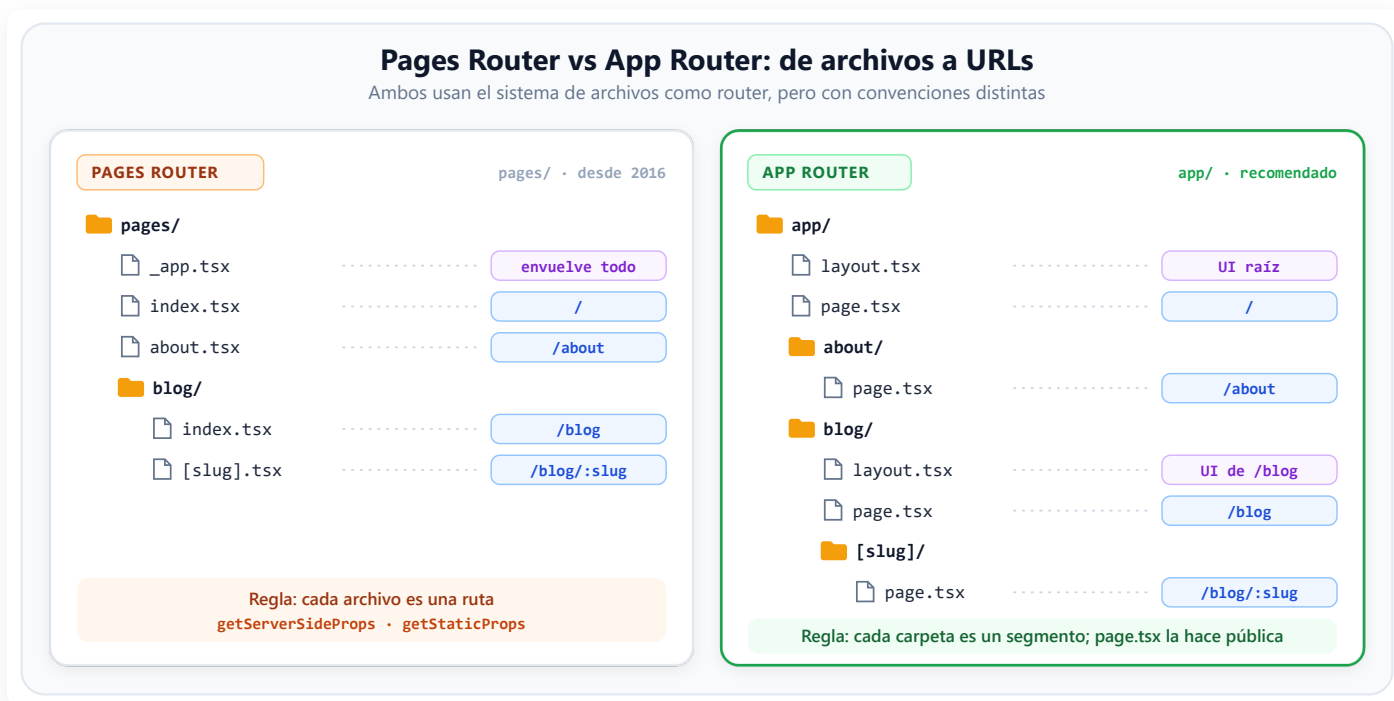
-  **Dashboards Privados:** Paneles tras login donde el SEO es irrelevante.
-  **Alta Interactividad:** Editores gráficos (canvas), suites SaaS, drag-and-drop.
-  **Estado Local Complejo:** Filtros y ordenamientos en memoria del navegador.
-  **APIs de Navegador:** Geolocalización, WebSockets, LocalStorage, Web Audio.

02. Pages Router vs App Router

El cambio fundamental de paradigma: del enrutamiento plano por archivos al enrutamiento por carpetas y componentes de servidor.

De Archivos a Carpetas: Evolución de Rutas

Estructura de Rutas: En Pages Router cualquier archivo era una URL pública. En App Router solo el archivo especial `page.tsx` es accesible, permitiendo colocar tests y componentes auxiliares libremente.



Comparativa Técnica: Pages vs App Router

Pages Router (`pages/`) — Legacy

- ✗ **Cualquier archivo es ruta:** Imposible colocar tests o utilidades en `pages/`.
- ✗ **Renderizado por página:** El layout se desmonta y re-renderiza al navegar.
- ✗ **Funciones monolíticas:** `getServerSideProps` bloquea toda la página.
- ✗ **Cascadas de red:** Todo o nada antes de pintar la UI.

App Router (`app/`) — Moderno

- ✓ **Ruta por carpeta:** Solo `page.tsx` es público; colocación libre de componentes.
- ✓ **Layouts preservados:** Mantienen estado sin re-renderizar entre páginas hijas.
- ✓ **React Server Components:** Servidor por defecto con 0 KB de JavaScript cliente.
- ✓ **Streaming con Suspense:** Carga por partes con `loading.tsx`.

Archivos Especiales y Suspense: Pages vs App Router

Evolución de Archivos: Pages Router dependía de `_app.tsx`, `_document.tsx` y `_error.tsx` monolíticos con renders bloqueantes; App Router introduce `layout.tsx` anidado, `page.tsx`, error boundaries y streaming con `loading.tsx` respaldado por React Suspense.

Archivos Especiales y Límites de Suspense: Pages Router vs App Router

Evolución de configuraciones monolíticas globales a envolturas modulares por segmento

PAGES ROUTER: MONOLITO EN `pages/`

`_app.tsx` **Envoltura Global Única**
Se re-renderiza y pierde estado al navegar entre rutas.

`_document.tsx` **HTML & Body Estático**
Solo corre en servidor; no admite lógica ni estilos dinámicos.

`_error.tsx` **Página de Error Global**
Pantalla monolítica sin granularidad contextual por módulo.

Limitación Central:

- Cero soporte de layouts anidados nativos (truco manual `getLayout`).
- Cero Suspense streaming: `getServerSideProps` bloquea toda la pantalla.
- Cualquier archivo `.tsx` dentro de `pages/` es expuesto como ruta.

APP ROUTER: ARCHIVOS ESPECIALES MODULARES

`layout.tsx` **Layout Anidado Persistente**
Preserva estado y no se desmonta al navegar entre páginas hijas.

`loading.tsx` **Límite Automático de `<Suspense>`**
Transmite skeletons instantáneos mientras el servidor hace streaming.

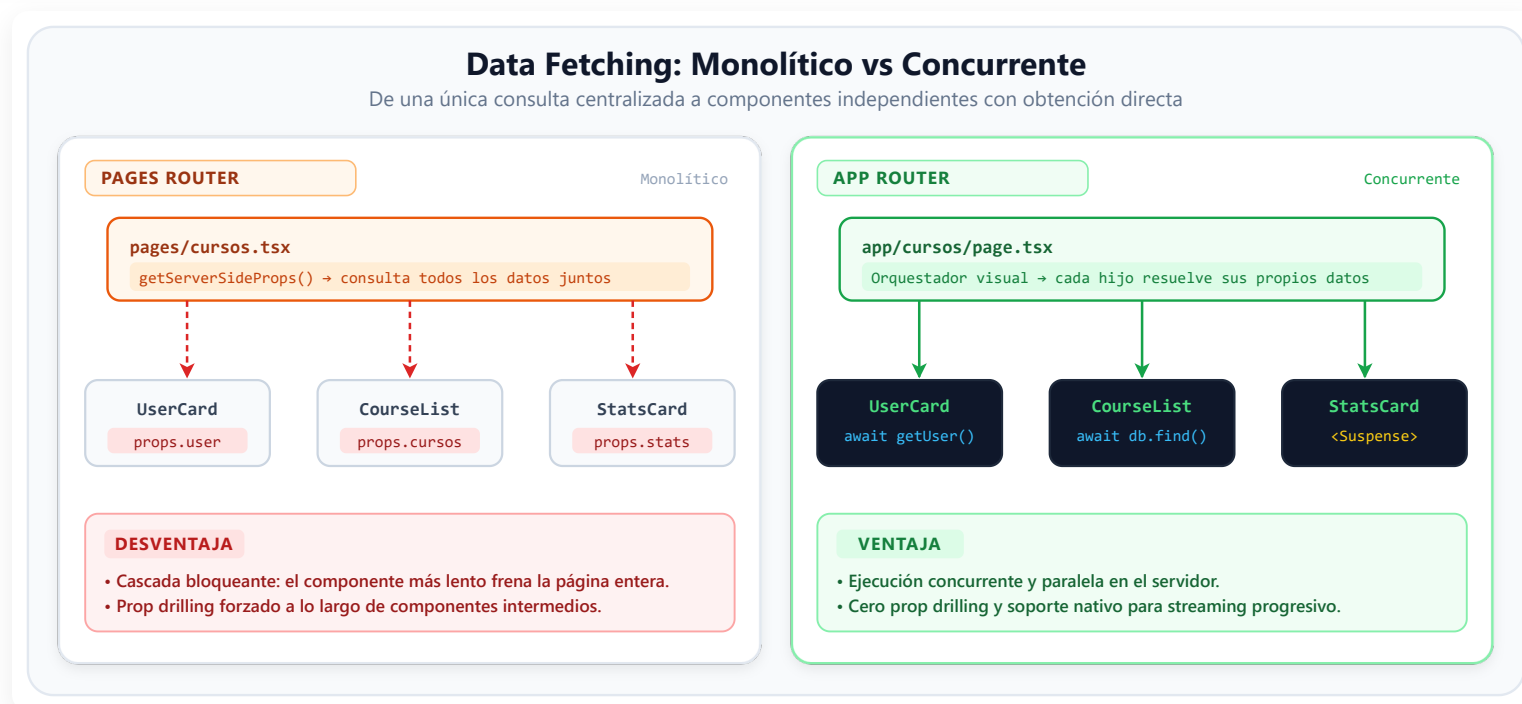
`error.tsx` **Error Boundary Contextual ('use client')**
Atrapa fallos sin tumbar el layout padre; permite retry con `reset()`.

Ventajas Clave:

- Cada carpeta es una isla aislada con su `layout`, `loading` y `error`.
- Streaming progresivo con React Suspense nativo de fábrica.
- Colocación privada: utilidades y tests conviven en la misma carpeta.

Evolución de la Obtención de Datos

Concurrencia de Datos: Transición de consultas monolíticas centralizadas (`getServerSideProps`) a componentes independientes con acceso directo a datos y streaming en el servidor.

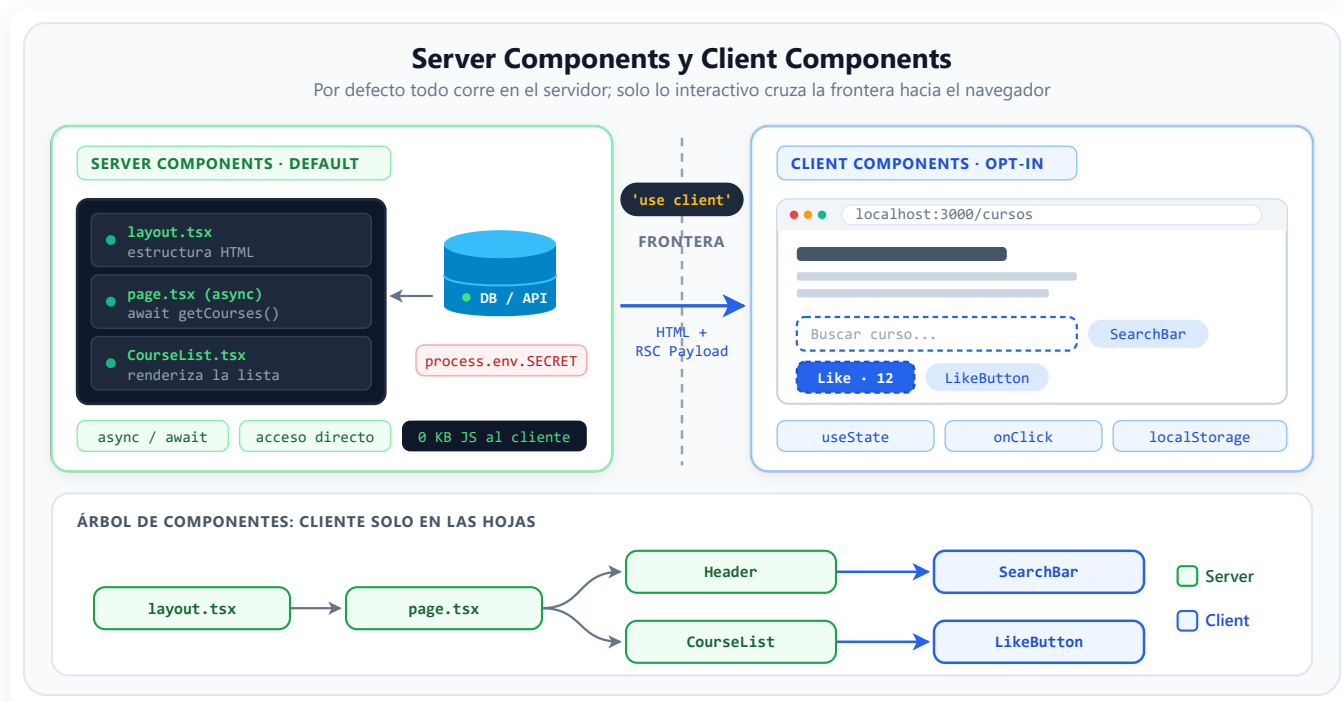


03. Server vs Client Components

El nuevo modelo mental: Servidor por defecto, bundle JS ligero (0 KB) y componentes interactivos aislados en la frontera.

Modelo Mental: Servidor vs Cliente

Distribución de Cómputo: Los Server Components procesan datos y lógica pesada en el servidor (0 KB de JS enviado al cliente); los Client Components se reservan para la interacción en el navegador.



Matriz de Decisión: Server vs Client Component

SERVER COMPONENT (POR DEFECTO)

-  **Acceso directo a recursos:** Base de datos (Prisma/TypeORM) y sistema de archivos.
-  **Secretos protegidos:** Tokens y API keys nunca viajan al navegador.
-  **0 KB bundle:** Librerías pesadas (Markdown, parseadores) no aumentan el peso JS.
-  **Componentes asíncronos:** `async/await` directo en la función del componente.

CLIENT COMPONENT ('USE CLIENT')

-  **Interactividad:** Escuchas de eventos como `onClick`, `onChange`, `onSubmit`.
-  **Hooks de estado y ciclo de vida:** `useState`, `useEffect`, `useReducer`.
-  **APIs del navegador:** `window`, `localStorage`, `document`, Web Audio.
-  **Hooks de navegación dinámica:** `usePathname()`, `useRouter()`.

La Directiva 'use client' como Frontera

En Next.js App Router, **todos los componentes son Server Components por defecto**. La directiva `'use client'` se coloca en la cima del archivo para declarar la frontera:

➔ Frontera del Grafo de Módulos

Cualquier componente o módulo importado dentro de un archivo con `'use client'` se convierte automáticamente en parte del bundle cliente.

✓ Hidratación Selectiva

Los Client Components aún se pre-renderizan a HTML en el servidor; solo añaden el script de hidratación para adjuntar eventos.

components/Counter.tsx (Cliente)

```
'use client';

import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => setCount(count + 1)}>
      Clicks: {count}
    </button>
  );
}
```

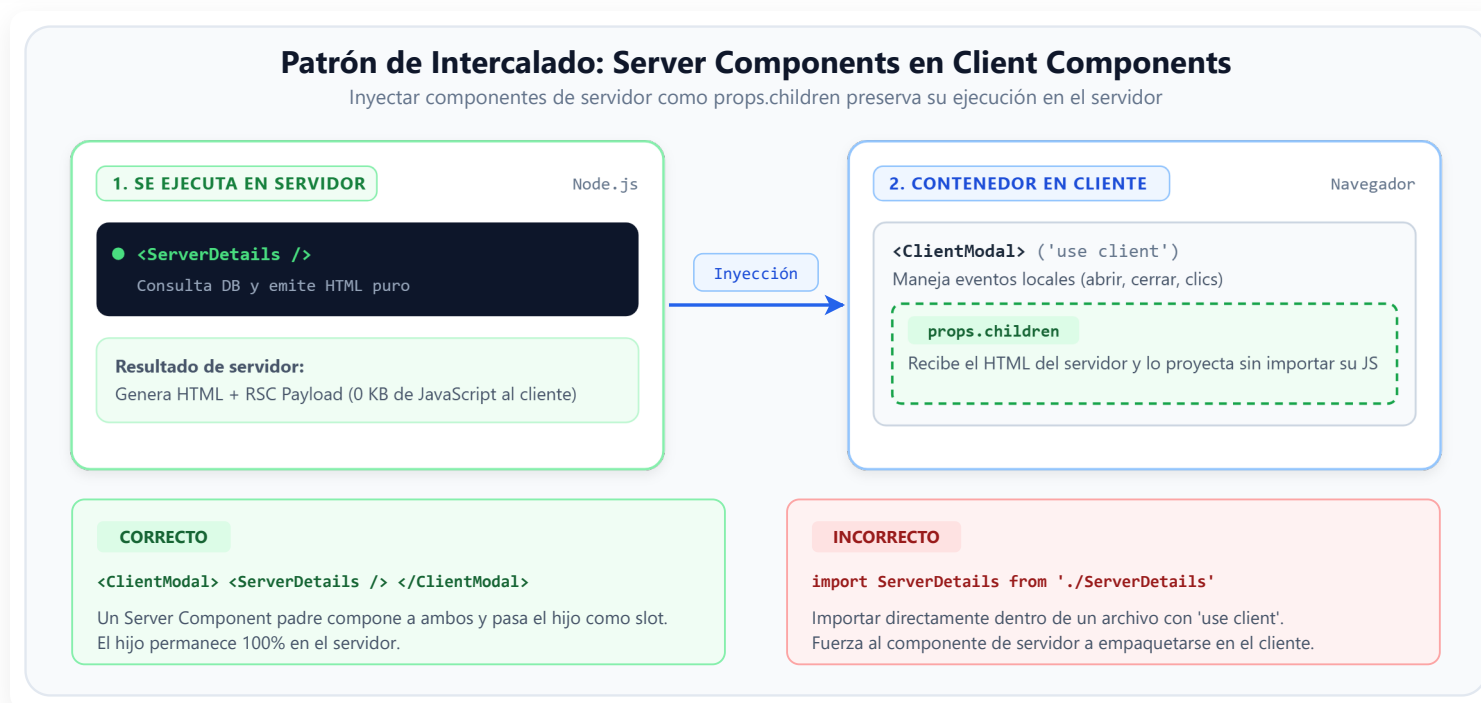
app/page.tsx (Servidor por defecto)

```
import Counter from '@components/Counter';

export default async function HomePage() {
  // Consulta directa a base de datos en el servidor
  const products = await db.product.findMany();
  return <main> <h1>Tienda </h1> <Counter /> </main>;
}
```

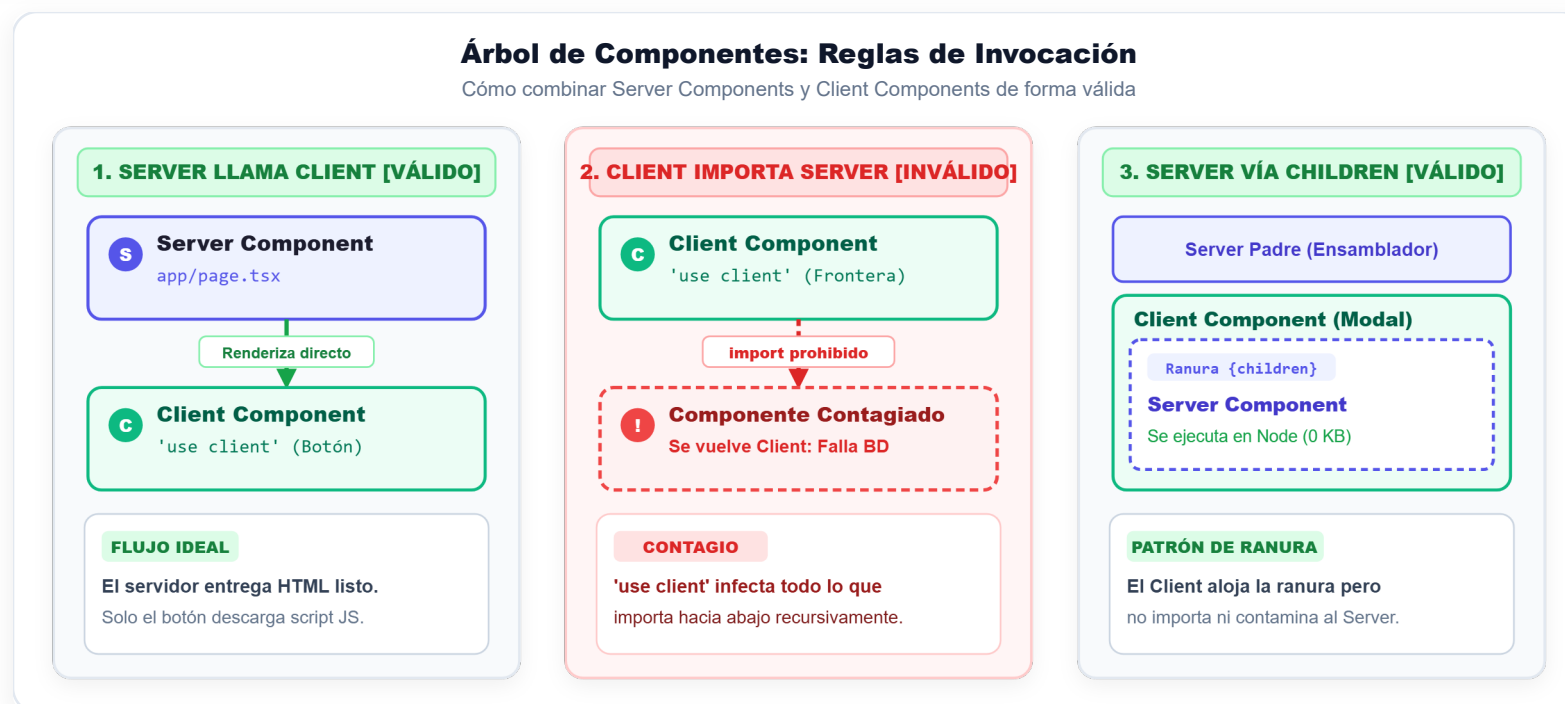
Patrón de Ranura (Interleaving): Servidor en Cliente

Patrón de Ranura: Un Client Component no puede importar un Server Component directamente, pero sí puede recibirlo como `children` proyectado desde un componente padre en el servidor sin perder sus privilegios.



Árbol de Componentes: Reglas de Invocación

Reglas de Composición: 1) Un Server puede renderizar un Client (Válido). 2) Un Client NO debe importar un Server (Antipatrón: contagia la frontera). 3) Para incrustar un Server dentro de un Client, pásalo como ranura `{children}` (Válido).



Regla 1 de Oro: Servidor por Defecto

✓ Todo Componente Nace en el Servidor

En Next.js App Router, asume que cada componente que creas es un **Server Component** hasta que la interactividad te exija lo contrario.

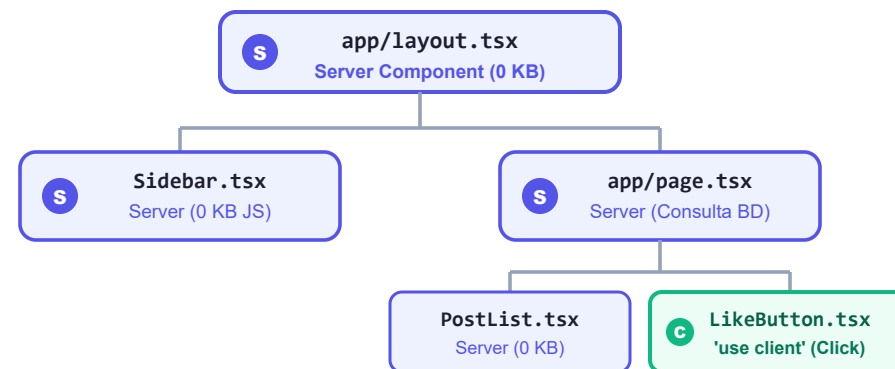
✓ **0 KB de JavaScript:** El código fuente del componente no viaja al navegador.

🗄️ **Acceso Directo:** Consultas directas a Prisma/TypeORM sin crear endpoints API intermedios.

✓ **Secretos Seguros:** Llaves privadas de autenticación permanecen protegidas en Node.js.

REGLA 1: SERVIDOR POR DEFECTO

Todo componente nace en el servidor hasta requerir eventos



ÁRBOL EN SERVIDOR (90%)

0 KB de JavaScript

Layout, Sidebar y Page corren en Node

FRONTERA CLIENTE (10%)

Solo LikeButton

Único script que hidrata el navegador

Regla 2 de Oro: Cliente en las Hojas

⚡ Evita el Contagio de la Frontera

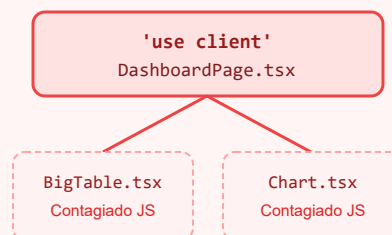
La directiva `'use client'` contagia recursivamente a todos sus módulos importados. No la uses en páginas ni en contenedores de alto nivel.

- ❌ **Anti-patrón:** Poner `'use client'` en `Dashboard.tsx` transforma todo su contenido en JS cliente.
- ✅ **Patrón Óptimo:** Conserva el contenedor en servidor y extrae solo el botón interactivo (`LikeBtn.tsx`) a cliente.
- 🕒 **Impacto:** Reduce bundles de 200 KB a menos de 2 KB en el navegador.

REGLA 2: CLIENTE EN LAS HOJAS

Aislar la interactividad en los nodos terminales para no contaminar el árbol

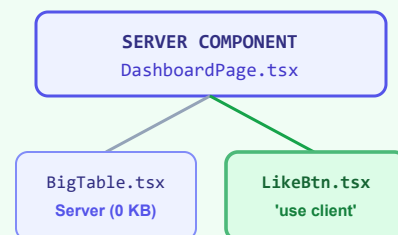
MAL: 'use client' EN LA RAÍZ



Consecuencia Negativa:

- Todo el árbol viaja en el bundle.
- Imposible consultar BD directo.
- Carga pesada para el navegador.

BIEN: SOLO LA HOJA ES CLIENTE



Beneficio Óptimo:

- 90% de UI renderizada en server.
- BigTable consulta DB directamente.
- Solo 2 KB de JS para el botón.

Regla 3 de Oro: Ranuras con Children

Inyección sin Acoplamiento (Patrón Ranura)

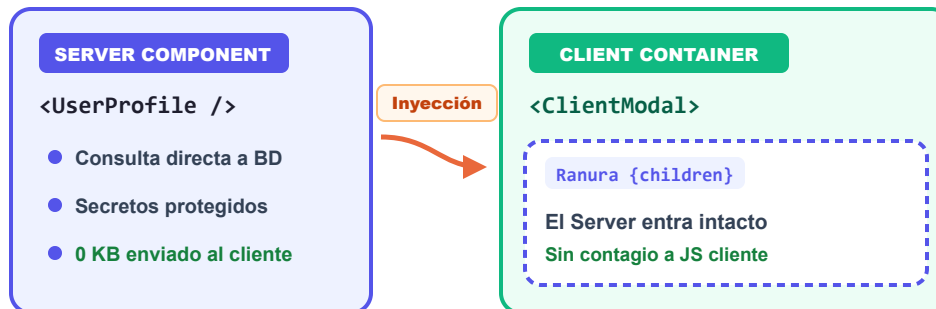
¿Cómo envolver un Server Component dentro de un contenedor interactivo (ej. un Modal animado)? Pásalo como prop `children`.

- ✓ **El Contenedor es la Ranura:** El componente cliente solo reserva el hueco con `{children}`.
- ✓ **Sin Importación Directa:** El cliente nunca importa el componente del servidor; el padre en servidor los une.
- ✓ **Cero Contagio:** El Server Component se resuelve en Node con 0 KB de JavaScript en el navegador.

REGLA 3: PATRÓN DE RANURA (SLOT)

Injectar Server Components como children dentro de Client Components

Servidor Padre (app/page.tsx): Ensambla ambos sin acoplar código



Client: Controla animación y eventos

Server: Mantiene acceso seguro a BD

ClientModal aloja el espacio (`{children}`) sin importar el código del servidor.

04. Arquitectura y Estructura

Anatomía del proyecto, archivos especiales, layouts anidados, metadata y navegación fluida del lado del cliente.

Inicialización del Proyecto con Create Next App

Para inicializar un proyecto Next.js moderno con soporte institucional, ejecutamos el CLI interactivo oficial:

```
npx create-next-app@latest mi-proyecto-web
```

✓ Opciones Recomendadas (Estándar 2026)

- ➔ **TypeScript:** Sí (tipado estricto en RSC y rutas).
- ➔ **Tailwind CSS:** Sí (utilidades atómicas con zero runtime).
- ➔ **App Router:** Sí (recomendado y moderno).
- ➔ **Import Alias:** `@/*` (apunta a la raíz del código).

🛡️ TypeScript en Next.js

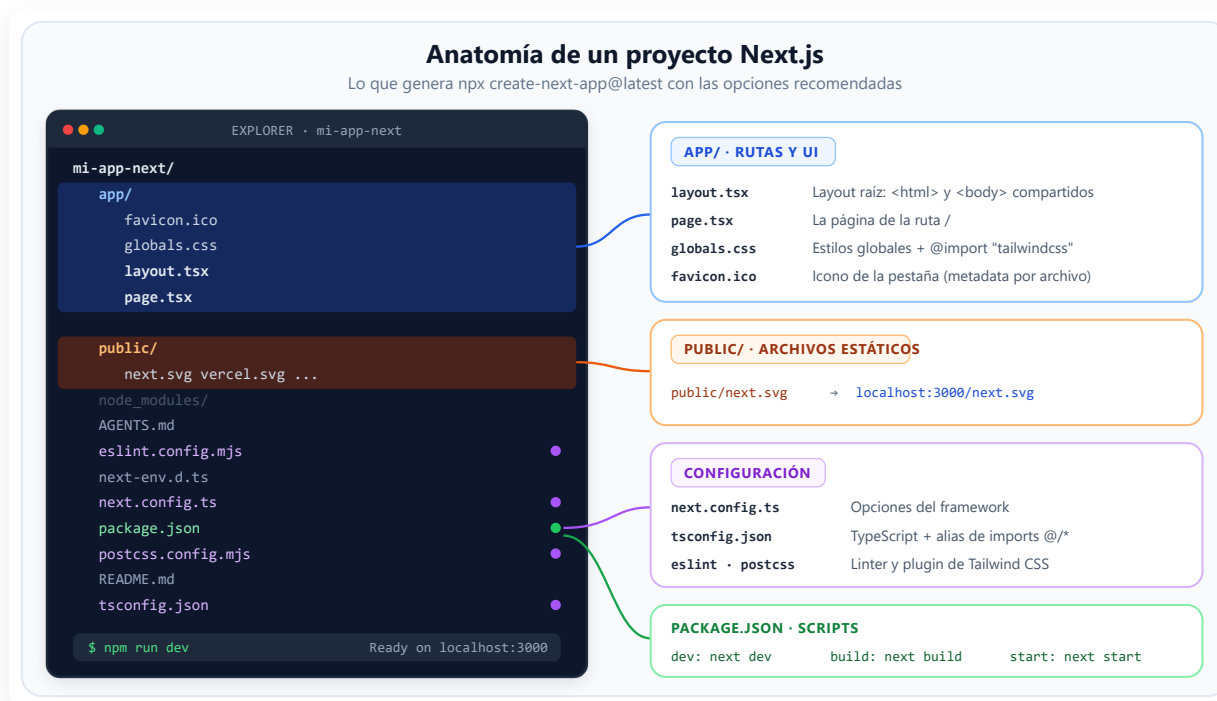
Next.js genera tipos automáticamente para parámetros de ruta dinámica (`params`) y metadatos, evitando errores tipográficos en URLs.

🕒 Tailwind CSS

Al compilar, Tailwind genera un archivo CSS minificado únicamente con las clases utilizadas, sin penalizar la velocidad de carga de la página.

Anatomía del Directorio del Proyecto

Organización del Proyecto: La carpeta `app/` aloja la jerarquía de rutas y componentes del App Router, mientras `public/` sirve recursos estáticos directamente desde la raíz del dominio.



Jerarquía de Archivos Especiales de Ruta

Cascada de Envoltura: En cada nivel de carpeta, Next.js encapsula la interfaz automáticamente: `layout` envuelve a `error`, este a `loading`, y finalmente se monta la página `page`.

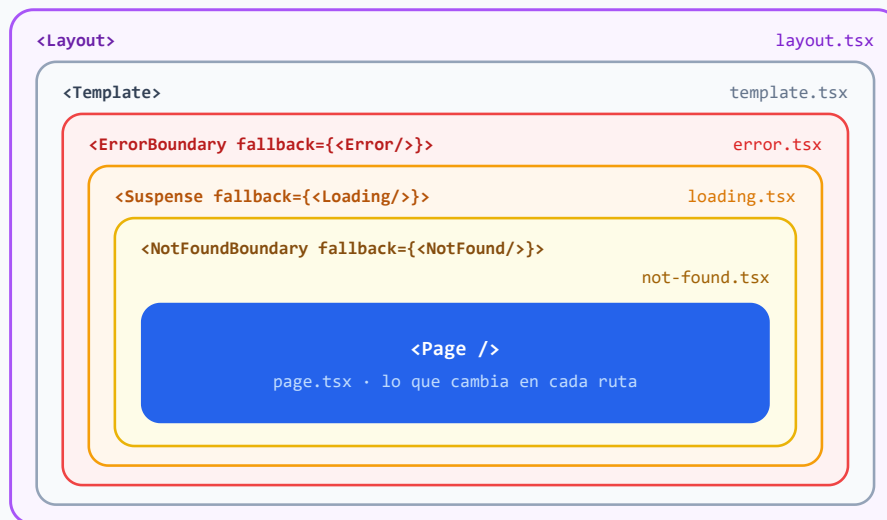
Archivos especiales: cómo se anidan al renderizar

Next.js envuelve tu `page.tsx` con los archivos especiales del mismo segmento, como muñecas rusas

APP/CURSOS/

- **layout.tsx**
UI compartida que persiste
- **template.tsx**
Como layout, pero se re-monta
- **error.tsx**
UI ante errores inesperados
- **loading.tsx**
Skeleton mientras carga
- **not-found.tsx**
UI para 404
- **page.tsx**
Contenido único de la ruta

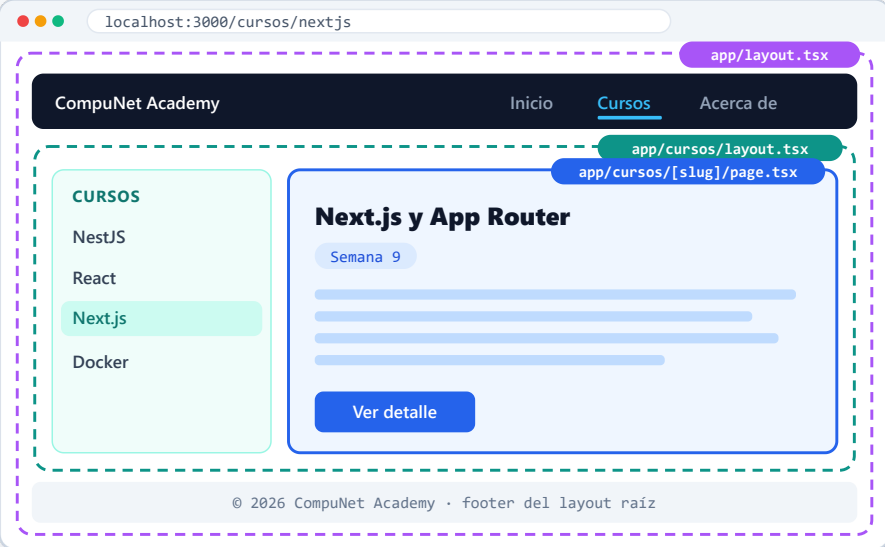
opcionales,
menos `page`



Layouts Anidados y Persistencia de UI

Persistencia de Estado: Al navegar entre rutas hijas, solo el segmento variable cambia; los layouts padre conservan su estado en memoria y no sufren re-renderizados innecesarios.

Layouts anidados: lo que ve el usuario en /cursos/nextjs
Cada carpeta puede aportar su propio layout; la página queda en el centro



AL NAVEGAR ENTRE CURSOS

/cursos/react → /cursos/nextjs

- Layout raíz (persiste)
- Layout de cursos (persiste)
- Página (cambia)

El estado del layout no se pierde

```
app/  
  layout.tsx  
  page.tsx  
  cursos/  
    layout.tsx  
    page.tsx  
    [slug]/  
      page.tsx
```

Metadata API: Optimización SEO Estática y Dinámica

✓ Metadata Estática (layout o page)

Exporta un objeto `metadata` con la configuración base del sitio o página.

```
import type { Metadata } from 'next';

export const metadata: Metadata = {
  title: 'Mi Aplicación Web',
  description: 'Plataforma de gestión Icesi',
};
```

⚡ Metadata Dinámica (generateMetadata)

Resuelve datos del servidor para generar títulos dinámicos en base a parámetros.

```
export async function generateMetadata({
  params
}: { params: { id: string } }): Promise<Metadata> {
  const p = await fetchProduct(params.id);
  return { title: p.name + ' | Tienda' };
}
```

Navegación del Lado del Cliente: <Link>

Navegación SPA: El componente `<Link>` realiza prefetching automático de rutas en el viewport y transiciones suaves sin recargar el documento HTML completo ni perder estado global.



Hooks de Navegación en Client Components

USEPATHNAME() — ENLACES ACTIVOS

Lee la ruta actual en componentes cliente para aplicar estilos visuales a la pestaña activa:

```
'use client';
import {usePathname} from 'next/navigation';
import Link from 'next/link';

export function NavLinks() {
  const pathname = usePathname();
  const active = pathname === '/dashboard';
  return <Link href="/dashboard" className={active ? 'active' : ''}>Inicio</Link>;
}
```

USEROUTER() — ACCIONES PROGRAMÁTICAS

Permite redireccionar o refrescar el árbol de renderizado tras enviar un formulario:

```
'use client';
import {useRouter} from 'next/navigation';

export function LoginForm() {
  const router = useRouter();
  const handleSuccess = () => {
    router.push('/dashboard'); // Redirección suave
    router.refresh(); // Refresca datos de RSC
  };
}
```